

DevAscend: Developer Growth Predictor

Oct 2, 2026 · Razi Ahmed

Project report, Master of Data Analytics. Submitted by Razi Ahmed (Roll No. 57), guided by Prof. Shivapradeep Muthupandi.

1. Abstract

DevAscend predicts a software developer's current level, market value and future growth, and recommends what to learn next. It is trained on 153,645 professional developers from the Stack Overflow Developer Surveys 2021–2025. The 360,519 raw responses are stored in Apache Cassandra.

Two supervised models were each chosen from three algorithms:

- **Level classifier (XGBoost):** predicts Junior, Mid, Senior or Lead with 65.2% accuracy, against 30.0% for guessing. 97.9% of its predictions are within one level.
- **Market value regressor (XGBoost):** predicts yearly salary with $R^2 = 0.649$ on log salary. MAE is \$27,810 and RMSE is \$52,049, against \$49,222 and \$74,183 for a no-ML baseline.

A growth simulator moves the user forward month by month to estimate the time to a goal. A hybrid recommender combines a 52-topic knowledge base, the ML what-if engine and five years of market trends into a learning roadmap with free resources. The system is delivered as a FastAPI backend and a Next.js web app with a 3D Three.js interface.

2. Introduction

Developers, especially juniors, cannot easily tell where they stand or what to learn next. DevAscend answers those questions with data from real developers instead of opinion.

2.1 Problem statement

A developer wants to know three things:

1. **Where am I?** Junior, Mid, Senior or Lead, and what is my market value?
2. **How long to my goal?** At my study pace, when do I reach Senior or a target salary?
3. **What should I learn next?** Which skills raise my level and pay, and which workplace skills am I missing?

No public dataset follows the same developers for many years. The project therefore has to learn from developers at different career stages and simulate one person's future.

2.2 Objectives

- Store five years of survey data in Apache Cassandra and train from it.
- Clean the data and define a fair Junior/Mid/Senior/Lead label.
- Train and compare three algorithms for each prediction task, and keep the best.
- Measure accuracy honestly on unseen data, including how far predictions are from actual values.
- Simulate growth month by month and estimate the time to a goal.
- Recommend skills, including ones the survey does not cover (Tailwind CSS, validation, system design), with free learning resources.
- Deliver everything as a web application with an API and a futuristic 3D interface.

2.3 Scope

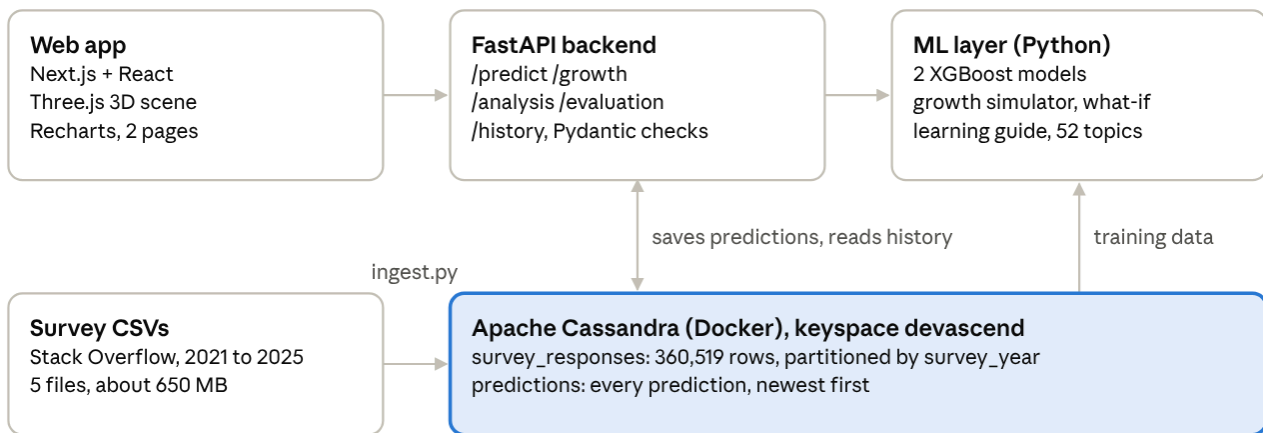
Inputs are years of coding, years of professional experience, role, skills, education, country, study hours per week and a goal (a level or a salary). Outputs are:

- level, 0–100 score and model confidence
- market value with an 80% range
- 3-year growth curve and time to goal
- skill gaps and what-if boosts
- learning guide and roadmap
- a model accuracy page

3. System architecture

The browser talks only to the FastAPI backend. The backend runs the ML layer in memory and uses Cassandra for prediction history. Training data flows from the survey files through Cassandra into the models.

One FastAPI service connects the 3D web app, the ML models and Cassandra



DevAscend architecture · web, API, ML, Cassandra

Offline pipeline: `ingest.py` loads the survey CSVs into Cassandra, then `clean.py` → `train.py` → `evaluate.py` → `trends.py`. Online: Web app → API → ML layer, and API ↔ Cassandra for history.

4. Dataset and Apache Cassandra

All 360,519 survey responses are loaded into Cassandra, and the training data is read back from Cassandra.

4.1 Dataset

The data comes from the [Stack Overflow Developer Survey](#) for 2021, 2022, 2023, 2024 and 2025 (ODbL licence). The CSV files were downloaded from the official Stack Overflow GitHub repository.

Survey column	Meaning	Stored as
YearsCode	total years of coding	years_code
YearsCodePro (2025: WorkExp)	professional years	years_code_pro
DevType	job role(s)	dev_type
Language/Platform/Webframe/Database/ToolsTech /MiscTech HaveWorkedWith	technologies used	six text columns
EdLevel, Country	education, country	ed_level, country

Survey column	Meaning	Stored as
ConvertedCompYearly	yearly salary in USD	salary_usd
AISelect (2023+)	uses AI tools?	ai_select
MainBranch, Employment	professional developer? employed?	filters

The survey changes a little each year. For example, 2025 renamed YearsCodePro to WorkExp and moved Docker from ToolsTech to Platform. `ingest.py` maps every year into one common schema.

4.2 Why Cassandra

Apache Cassandra is a distributed NoSQL database with no master node. It scales horizontally and has no single point of failure. In DevAscend it runs as one node in Docker (`docker-compose.yml`) and has two jobs:

1. **Dataset store:** the training data comes from it.
2. **Live application database:** every prediction is saved and the history is read back.

4.3 Tables (keyspace `devascend`)

Table	Primary key	Query it serves
survey_responses	((survey_year), response_id)	all developers of one survey year
predictions	((bucket), created_at DESC, id)	the latest N predictions, newest first

Cassandra design is query-first: there are no JOINS, so each table is shaped around the query it answers. The **partition key** decides which node stores a row, and rows with the same key are stored together. The **clustering key** sorts rows inside a partition.

4.4 Loading and reading

- `ingest.py` uses a prepared INSERT, parsed once and reused for every row, and runs 100 inserts in parallel (`execute_concurrent_with_args`). The 360,519 rows load in about four minutes.
- `clean.py` reads one partition per survey year with paging (`fetch_size = 5000`).
- Row counts are taken per partition. A `COUNT(*)` over the whole table is a full cluster scan, and it timed out during development.

- On Windows with Python 3.12+, the Cassandra driver needs the `pyasyncore` package, because Python removed the `asyncore` module it depends on.

5. Data cleaning, features and the level label

Cleaning keeps 153,645 of 360,519 responses. Each kept developer becomes 151 numeric features.

5.1 Cleaning steps (`clean.py`)

Step	Rows left
Raw responses, 5 years	360,519
Professional developers, employed, salary given	163,791
Salary between \$3,000 and \$600,000 (typos and outliers removed)	156,922
Valid experience ("Less than 1 year" → 0, "More than 50 years" → 51)	155,862
Has a developer role (students, retired, academics removed)	154,797
At least one skill	153,645

Other transformations:

- **Role:** more than 30 DevType answers are mapped to 15 roles with a priority list. Examples are Full-stack, Back-end, Front-end, Mobile, Data/ML, DevOps/Cloud, Manager and Architect.
- **Education:** converted to an ordered number from 0 (primary) to 6 (PhD/professional). Numbers keep the order, so Master's is above Bachelor's.
- **Skills:** the six technology columns are merged and spellings unified, for example "React.js" → "React" and "AWS" → "Amazon Web Services (AWS)". Package managers such as npm and Homebrew are ignored.
- **Consistency:** years coding is never below professional years.

5.2 Features (`features.py`)

Group	Columns	Encoding
years_code, years_pro, ed_level, survey_year, num_skills	5	numbers
Role	15	one-hot
Country (top 30 + Other)	31	one-hot
Skills (100 most common)	100	multi-hot (1 = uses it)

Training and the API share the same `build_features` function, so a user's profile is encoded exactly like the training data.

5.3 The level label

The survey has no "Junior/Senior" question, so the label is defined from two facts:

$$\text{level_score} = 100 \times \frac{\text{experience percentile} + \text{salary percentile}_{\{\text{same country, same year}\}}}{2}$$

The salary percentile is computed within each country and survey year, which makes the label fair across countries and inflation. The score is then cut into four levels:

Level	Share	level_score range	Median professional years	Median salary
Junior	30%	below 34.9	3	\$36,410
Mid	30%	34.9 – 59.2	7	\$62,820
Senior	25%	59.2 – 77.7	12	\$90,647
Lead	15%	77.7 and above	20	\$139,216

The 0–100 score shown to users is the probability-weighted typical score of each level: $P(\text{Junior}) \cdot 20 + P(\text{Mid}) \cdot 48 + P(\text{Senior}) \cdot 68.2 + P(\text{Lead}) \cdot 84.7$.

6. Models and training

XGBoost was the best of three algorithms for both tasks, and both winners are saved with `joblib` (`train.py`).

6.1 Training setup

- **Split:** 80% train (122,916 developers) and 20% test (30,729), stratified by level, `random_state = 42`. All six models use the same split, so the comparison is fair.
- **Model A, the level classifier:** classification into 4 classes. The best model is chosen by macro F1.
- **Model B, the market value regressor:** regression on yearly salary, used as a measurable stand-in for how the market values a profile. It learns $\log(\text{salary})$, because salaries are right-skewed, and predictions are converted back with $\exp()$.
- **Monotone constraint:** XGBoost is told that predicted salary may only rise when years of experience rise. This builds domain knowledge into the model and keeps the growth curve logical.

6.2 Algorithms compared

Algorithm	How it works	Settings
Logistic / Linear Regression	weighted sum of the features; a linear baseline	StandardScaler first; C = 0.5 for logistic
Random Forest	150 decision trees on random samples, combined by voting or averaging (bagging)	max depth 18, min 5 samples per leaf
XGBoost	trees built one after another, each correcting the previous errors (gradient boosting)	500–800 trees, depth 6–7, learning rate 0.06–0.08

6.3 Results on the test set

Model A: level	Accuracy	F1 (macro)	Within ± 1 level
Logistic Regression	62.2%	0.603	97.9%
Random Forest	63.3%	0.623	97.9%
XGBoost (selected)	65.2%	0.646	97.9%

Model B: market value	R ² (log)	MAE	RMSE
Linear Regression	0.584	\$31,089	\$56,327
Random Forest	0.633	\$28,401	\$52,959

Model B: market value	R ² (log)	MAE	RMSE
XGBoost (selected)	0.649	\$27,810	\$52,049

XGBoost wins because boosting captures non-linear effects and interactions. For example, Kubernetes is worth more in some countries and roles. It also handles many sparse 0/1 skill columns well.

6.4 Feature importance

- **Level model:** years of professional experience dominates. Skills such as AWS, Kubernetes, Terraform and PHP follow, along with the Manager and Architect roles.
- **Value model:** country dominates (USA, India, Switzerland and others), then experience, then skills.

7. Evaluation and results

On 30,729 unseen developers, both models clearly beat a no-ML baseline. Level predictions are almost always within one level, and salary predictions have a 25.5% median error with a reliable 80% range (`evaluate.py`).

7.1 Level classifier

Metric	Value	Meaning
Accuracy	65.2%	exact level (always guessing the most common level gives 30.0%)
F1 (macro)	0.646	balance of precision and recall, averaged over 4 levels
Within ±1 level	97.9%	right level or a neighbouring one
Off by 2+ levels	2.1%	large mistakes
Mean level error	0.37	average distance in levels

Confusion matrix (% of each actual level; rows = actual, columns = predicted):

Actual \ Predicted	Junior	Mid	Senior	Lead
Junior	83%	17%	1%	0%
Mid	18%	58%	21%	3%

Actual \ Predicted	Junior	Mid	Senior	Lead
Senior	1%	27%	54%	19%
Lead	0%	3%	32%	65%

Almost every mistake is one level away. That is expected, because the levels are cut points on a continuous score. Mid and Senior are hardest because they have neighbours on both sides.

7.2 Market value regressor

Metric	XGBoost	Baseline (median salary)
R ² on log salary	0.649	0
R ² in dollars	0.482	0
MAE	\$27,810	\$49,222
RMSE	\$52,049	\$74,183
Median absolute error	\$15,451	
Median % error	25.5%	
MAPE	46.0%	
Within ±20% of real salary	40.6%	
Bias (mean predicted – actual)	–\$8,192	

$$\text{MAE} = \frac{1}{n} \sum |\hat{y}_i - y_i| \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum (\hat{y}_i - y_i)^2} \quad R^2 = 1 - \frac{\sum (\hat{y}_i - \bar{y})^2}{\sum (y_i - \bar{y})^2}$$

RMSE is much larger than MAE because squaring makes a few very large misses on high salaries count heavily.

7.3 How predictions differ from actual data

Real salary band	Avg real	Avg predicted	Median % error
under \$25k	\$13,469	\$24,229	58%
\$25k–50k	\$38,183	\$44,675	27%

Real salary band	Avg real	Avg predicted	Median % error
\$50k–100k	\$73,103	\$74,085	21%
\$100k–150k	\$123,619	\$115,033	21%
over \$150k	\$224,129	\$151,082	27%

Low salaries are over-predicted and very high salaries are under-predicted. This is regression to the mean: the survey does not record company, city or negotiation, so extreme cases look more average to the model. Typical salaries (\$50k–150k) are the most accurate.

Uncertainty range: on test data, 80% of real salaries were between $0.575\times$ and $1.783\times$ the prediction, and exactly 80.0% of test developers fell inside. The app shows this as “likely \$X – \$Y”.

8. Growth simulator and what-if engine (Model C)

Model C is logic on top of Models A and B, not a separately trained model. It moves the user forward one month at a time and re-predicts (`predictor.py`).

8.1 Simulation loop

1. For month $m = 0 \dots 120$, set experience to its starting value plus $m/12$ years.
2. Add the next roadmap skill once its study hours are done. At 10 h/week a user studies about 43 hours per month; topic hours come from the knowledge base (Tailwind 15 h, React 60 h, default 80 h).
3. Predict level probabilities (Model A) and salary (Model B) for all 121 months in one batch.
4. The goal is reached at the first month where $P(\text{level} \geq \text{goal}) \geq 50\%$, or where predicted salary $\geq$ the target.

Two scenarios are simulated, **experience only** and **with the roadmap**. The gap between them is the value of learning.

Smoothing:

- **Interpolation:** the survey records whole years, so the tree models only change at year boundaries. 2.25 years is predicted as 75% of the 2-year prediction plus 25% of the 3-year prediction.
- **Running maximum:** gaining experience should not make a developer worse, so each month keeps at least the previous month's value.

Example: a Junior front-end developer in India with HTML/CSS, JavaScript and React, studying 10 h/week. The simulation estimates Senior in about 7 years 2 months with the roadmap, against 8 years 9 months with experience only. That is 19 months sooner.

8.2 What-if engine

1. Candidates are skills the user lacks that at least 5% of Senior/Lead developers in their role use.
2. Each candidate is switched on in the profile and both models are re-run.
3. The change in market value (%) and level score is measured.
4. Skills are ranked by impact = (value % + score change) × (0.5 + share of seniors using it). The weight keeps rare niche tools from outranking widely used skills.
5. For the top 8 skills, the engine also reports how many months sooner the goal is reached.

This is a simple form of model explanation, similar in spirit to SHAP: how does the prediction change if one feature changes?

8.3 Skill gap analysis

For each role and level, training computed the share of developers using each skill. The gap list shows skills that at least 10% of higher-level developers in the user's role use and the user does not have.

9. Learning guide: a hybrid recommender

The survey knows only about 100 technologies, so pure ML cannot recommend Tailwind, form validation or system design. DevAscend combines three sources (`recommender.py`).

Source	What it contributes	Example
Knowledge base (52 topics, <code>learning_topics.json</code>)	prerequisites, role, level, why it matters, free resources	HTML/CSS unlocks Tailwind CSS
ML what-if engine	predicted market value boost for this profile	TypeScript: +7.9%
Market trends (<code>trends.py</code> , 5 survey years from Cassandra)	rising or declining skills	TypeScript 38% → 50% of developers

Using knowledge where data is missing is the standard answer to the **cold-start problem**.

9.1 Filtering and scoring

A topic is a candidate when the user does not know it, at least one prerequisite is known, it fits the role, and it is at most one level above the user. Each candidate is scored:

Rule	Points
core skill of the role (e.g. ML fundamentals for Data/ML)	+3
relevant to the role, not core	+1
workplace essential and the user is Junior or Mid	+2
rising in survey data (knowledge base only: +1)	+1.5
declining in survey data	-1
ML market value boost (impact ÷ 10, max 3)	0 to +3
typically learned at the user's level	+0.5

Every recommendation lists its reasons, for example “You know HTML/CSS → this is a natural next step”.

9.2 Roadmap as a path through the skill tree

The roadmap is built greedily: pick the best topic, treat it as learned (which can unlock new topics, such as React → Next.js), and repeat up to 8 topics, at most 3 of them essentials.

Profile (1 year experience)	Roadmap
Front-end: HTML/CSS, JavaScript	TypeScript → Tailwind CSS → React → Next.js → Form validation → Accessibility → Testing → SQL
Back-end: Java, SQL	Redis → Go → REST API design → PostgreSQL → Docker → System design → Server-side validation → DB design
Data/ML: Python, Pandas, SQL	LLM features → PostgreSQL → ML fundamentals → Testing → Git workflow → Env/secrets → Linux → Docker

9.3 Market trends from the data

- **Rising (2021 → 2025):** TypeScript 38% → 50%, Python 41% → 54%, PostgreSQL 41% → 50%, Docker 54% → 62%, Go 11% → 18%, Rust 6% → 14%, FastAPI 3% → 12%.
- **Declining:** jQuery, Heroku, MySQL, .NET Framework, Angular.
- **AI tool use among professional developers:** 43% (2023) → 63% (2024) → 81% (2025), from the AISelect question stored in Cassandra.

All 100+ resource links (official docs, MDN, freeCodeCamp, the System Design Primer, Chai aur Code, CodeWithHarry and others) were checked automatically.

10. Backend API (FastAPI)

The API loads both models once at startup and answers a full analysis in about one second (`api/main.py`). Interactive documentation is at <http://localhost:8000/docs> .

Method	Path	Returns
GET	/health	API status, models loaded, Cassandra connected
GET	/options	roles, countries, skills and education levels for the form
GET	/model-info	metrics of all six algorithms
GET	/evaluation	full test-set report: metrics, confusion matrix, errors by band, real examples
POST	/predict	level, score, probabilities, salary, salary range, confidence
POST	/growth	36-month curve, checkpoints, time to goal; saves the prediction to Cassandra
POST	/analysis	skill gaps, what-if boosts, roadmap, learning guide
GET	/history	latest predictions from Cassandra

- **Validation:** Pydantic models (`schemas.py`) reject bad input with a 422 error. Examples are years outside 0–50 or an unknown goal type.
- **CORS:** allows the Next.js site on any localhost port to call the API.
- **Resilience:** if Cassandra is down, predictions still work; only history is disabled.

Example request to `/predict` :

```
{
  "years_code": 4,
  "years_pro": 2,
  "role": "Full-stack",
  "country": "India",
  "ed_level": 4,
  "skills": ["JavaScript", "React", "Node.js"]}

```

11. Frontend (Next.js, React, Three.js)

The web app has two pages, Predictor and Model accuracy, wrapped in a futuristic 3D interface (`web/`).

- **Reusable components:** `Form` handles submit, loading and errors. `Input` renders a label plus a text, number or select field and calls `onChange(name, value)`, so one handler updates any field.
- **Profile form:** includes a skill picker with search and chips, and a “Fetch from GitHub” button that reads a user’s public repositories through the GitHub REST API.
- **Dashboard:** level ring and probabilities, time-to-goal cards, a Recharts growth chart with two scenarios on one axis, skill-gap bars, what-if table, roadmap timeline with resource links, learning guide, and history from Cassandra.
- **Accuracy page:** metric tiles, model vs baseline, confusion-matrix heat grid, errors by salary band, country and level, and 12 real test developers with predicted vs actual values.
- **3D design:** a Three.js scene sits behind the interface. It shows a glowing wireframe crystal, orbiting rings, a particle star field and a rising grid that react to the mouse. The scene uses glass-style panels, a neon cyan and violet palette, and respects reduced-motion settings.

12. Installation and how to run

Requirements: Python 3.11+, Node.js 18+ and Docker Desktop. Steps 1–3 are one-time; steps 4–6 are needed every time.

1. Create the Python environment: `python -m venv .venv`, then `.venv\Scripts\pip install -r ml\requirements.txt -r api\requirements.txt`.
2. Download the five survey CSVs into `ml\data\raw\survey_<year>.csv`.
3. Run `docker compose up -d`, then in `ml\` run `python ingest.py`, `python clean.py` and `python train.py`. Training also runs the evaluation and trend analysis.
4. Start Cassandra: `docker compose up -d`.
5. Start the API: in `api\` run `..\venv\Scripts\python -m uvicorn main:app --port 8000`.
6. Start the website: in `web\` run `npm install` (first time), then `npm run dev`, and open `http://localhost:3000`.

13. Limitations, future work and conclusion

13.1 Limitations

- **Simulated future:** the data is cross-sectional, so the same person is never observed over years.
- **Correlation, not causation:** for example, Kafka users may earn more partly because Kafka is used at large companies.
- **Salary as a proxy:** pay also depends on company, city and negotiation, which the survey does not record.
- **Estimates:** study hours per topic come from the knowledge base, not from data.
- **Knowledge-base-only skills:** topics such as Tailwind take study time in the simulation but cannot change the ML prediction.
- **Survey bias:** respondents are not a perfect sample of all developers.

13.2 Future work

- Collect data that follows the same users over time.
- Add city and company size as features.
- Add SHAP explanations.
- Tune hyper-parameters with cross-validation.
- Use quantile regression for better ranges.
- Run Cassandra as a three-node cluster with replication factor 3.

13.3 Conclusion

DevAscend shows a complete ML system: data stored and served from Cassandra, a fairly defined label, a model comparison on a shared split, honest evaluation against baselines and real values, and a simulation that turns static predictions into a growth plan. The hybrid recommender fills the gap that the data alone cannot cover, giving junior developers concrete next steps with free resources.

14. References

1. Stack Overflow Developer Survey 2021–2025, survey.stackoverflow.co, data in [StackExchange/Survey on GitHub](https://stackexchange.com/survey-on-github).
2. [Apache Cassandra documentation](https://cassandra.apache.org/doc/latest/).
3. [scikit-learn user guide](https://scikit-learn.org/stable/user_guide.html).
4. [XGBoost documentation](https://xgboost.ai/doc/).

5. [FastAPI documentation](#).
6. [Next.js documentation](#).
7. [Three.js documentation](#).
8. Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.
9. Breiman, L. (2001). Random Forests. Machine Learning 45, 5–32.